bench4JSBACH Documentation

Release 1

Deike Kleberg

September 13, 2016

Contents

1 Introduction **2**
- 1.1 Whole department knowhow for your model evaluation . . . 2
- 1.2 Status . . . 2

2 Report **2**
- 2.1 General Remarks . . . 2
- 2.2 Examples . . . 3
- 2.3 Structure . . . 3
- 2.4 Main Part . . . 4
- 2.5 Summary . . . 6

3 Running **6**
- 3.1 Getting Started . . . 6
- 3.2 Running . . . 7
- 3.3 Experiments . . . 8
- 3.4 References . . . 8

4 Participate **9**
- 4.1 Introducing Features . . . 9
- 4.2 Diagnostic Fields . . . 10
- 4.3 Configuration . . . 10
- 4.4 I/O Dependencies . . . 11
- 4.5 Processing . . . 11

5 Runtime **11**
- 5.1 Fetching Data . . . 11

6 Configuration **12**
- 6.1 Configuration API Reference . . . 12
- 6.2 Example configuration for the report section of the diagnostic field ‘albedo´ . . . 15

7 Indices and tables **16**

Contents:

Introduction

The tool **bench4jsbach** is being developed in order to make the comparison of **JSBACH** land model runs with one another and with data products straight forward and user friendly. For this purpose it provides a framework to separate the associated tasks

1. fetch, select and interpolate data
2. perform analyses, benchmarks and plotting
3. document the results in a report

It is capable to cope with multiple experiments, candidates as well as references. Each candidate experiment is compared with all references, pairwise or as a group depending on the diagnostic.

bench4jsbach is flexible and extensible to grow inline with requirements. It is written in object oriented Python and uses specialized classes for the tasks enumerated above.

Whole department knowhow for your model evaluation

From the guiding vision ...

- make the most significant diagnostic usable for the whole department
- solve technical issues once, in particular concerning data retrieval and preparation

... some steps ...

- enable the groups to contribute their diagnostic
- develop/use a library for diagnostic, benchmark and plotting

... to the usable potential

- support model development
- make model benchmarking output easy to use

Status

The basic functionality of **bench4jsbach** is available. Placed at the disposal:

experiment type:

- MPIESM1

diagnostic field:

- albedo
- hydrology

Report

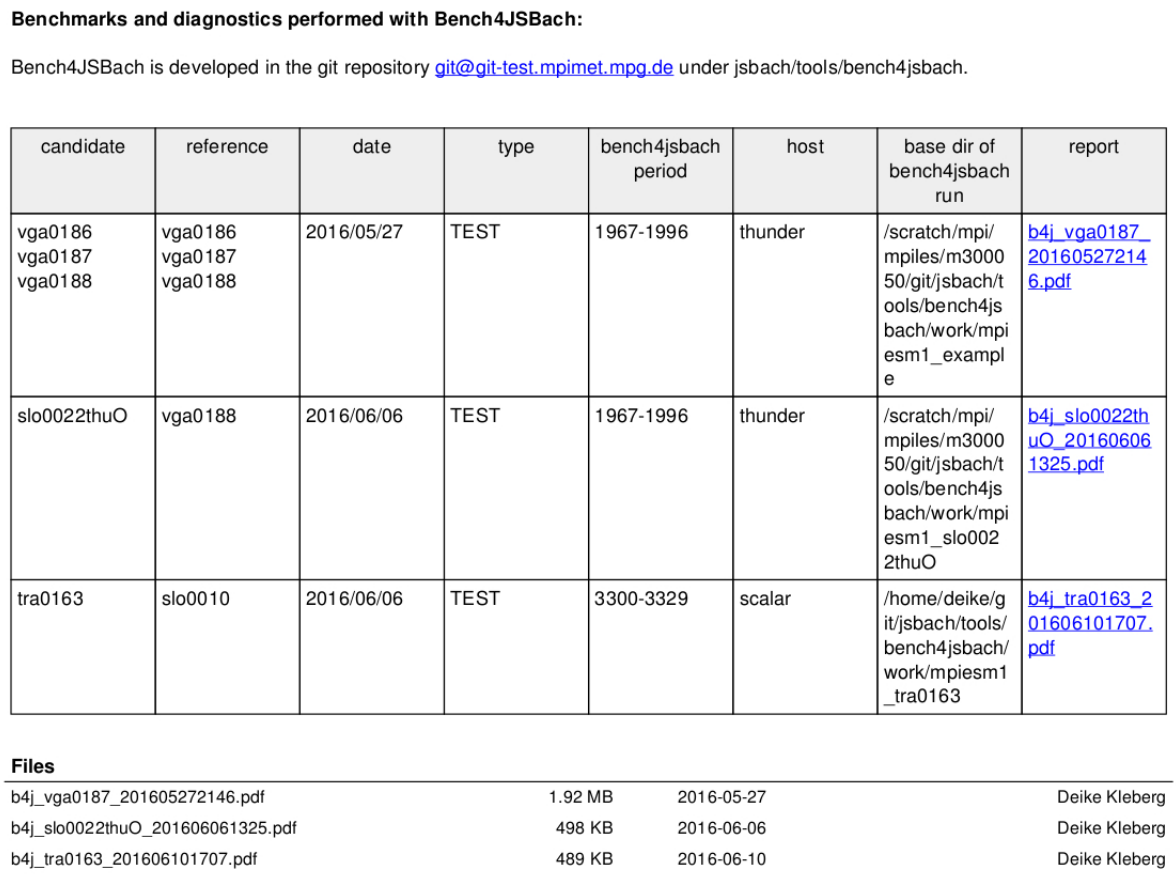
General Remarks

- pdf generation automatically
- switchable

- typesetting system LaTeX

Examples

Examples of **bench4jsbach** generated reports can be found on the wikipage Bench4JSBACH simulations. The figure below only displays an old status, to get an up-to-date impression follow the link in the caption and open one of the listed reports.



Benchmarks and diagnostics performed with Bench4JSBach:

Bench4JSBach is developed in the git repository git@git-test.mpimet.mpg.de under jsbach/tools/bench4jsbach.

candidate	reference	date	type	bench4jsbach period	host	base dir of bench4jsbach run	report
vga0186 vga0187 vga0188	vga0186 vga0187 vga0188	2016/05/27	TEST	1967-1996	thunder	/scratch/mpi/mpiles/m300050/git/jsbach/tools/bench4jsbach/work/mpiesm1_example	b4j_vga0187_201605272146.pdf
slo0022thuO	vga0188	2016/06/06	TEST	1967-1996	thunder	/scratch/mpi/mpiles/m300050/git/jsbach/tools/bench4jsbach/work/mpiesm1_slo0022thuO	b4j_slo0022thuO_201606061325.pdf
tra0163	slo0010	2016/06/06	TEST	3300-3329	scalar	/home/deike/git/jsbach/tools/bench4jsbach/work/mpiesm1_tra0163	b4j_tra0163_201606101707.pdf

Files

b4j_vga0187_201605272146.pdf	1.92 MB	2016-05-27	Deike Kleberg
b4j_slo0022thuO_201606061325.pdf	498 KB	2016-06-06	Deike Kleberg
b4j_tra0163_201606101707.pdf	489 KB	2016-06-10	Deike Kleberg

Fig. 1: **Figure:** Bench4JSBACH simulations, detail

Structure

The reports generated by **bench4jsbach** contain three sections: introduction, main part and summary.

1. **Introduction** This is the overall **bench4jsbach** run description. It consists of
   (a) all experiments table
   (b) experiment subsections
      i. table of metadata
      ii. description of special features
   (c) all diagnostic fields table (methods, benchmarks)

(d) system and runtime information

2. **Main part** contains one section for each diagnostic field. They are structured as follows
   (a) field description
   (b) method subsections
      i. scientific information
      ii. evaluation criteria for benchmarks
      iii. technical information
      iv. presentation of results e.g. plots
3. **Summary** display of benchmark results from all diagnostic fields

Main Part

The main part is grouped by the diagnostic fields and their methods. The methods are scientifically and technically explained and evaluation criteria for the benchmarks are supplied.

We have selected some extracts exemplarily, displayed in the four figures below.

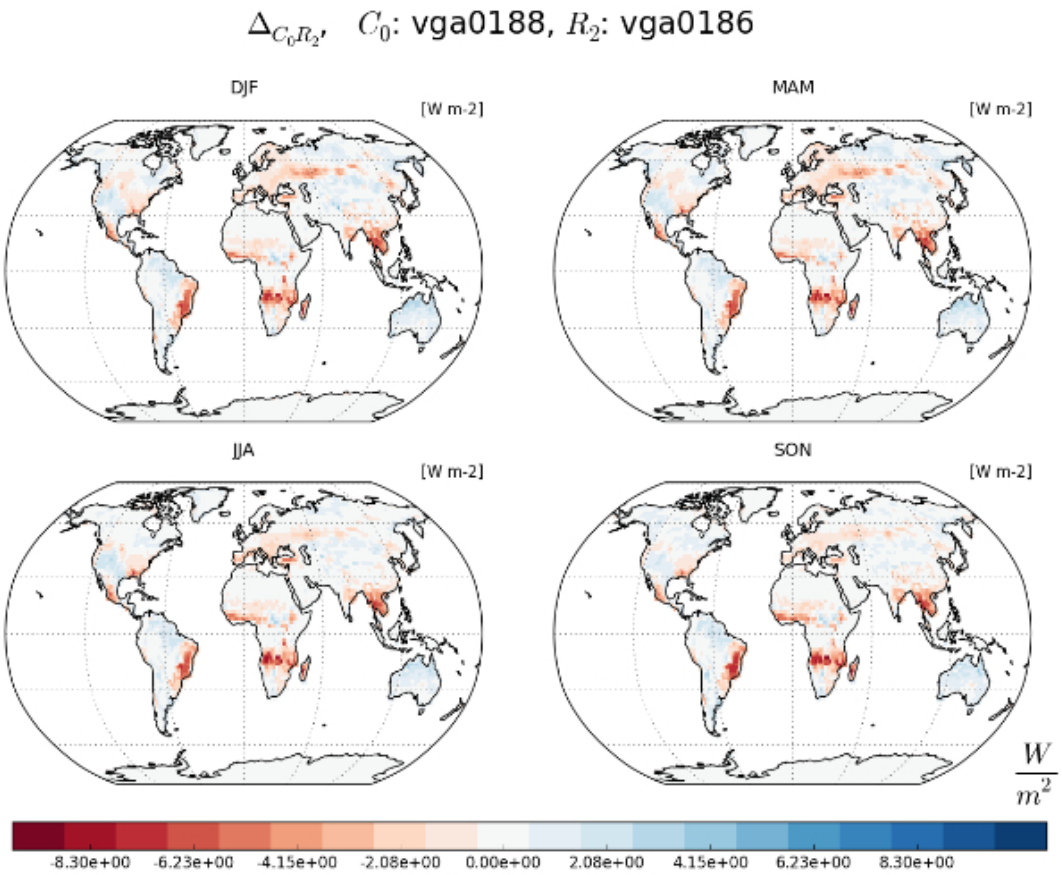


Fig. 2: **Figure:** difference plot of seasonal mean, example

2.1 Seasonal mean difference

The difference between candidate and reference experiments albedo seasonal means is weighted with the reference experiments downwelling radiation seasonal means:

$$\begin{aligned} \Delta_{CR} &= (A_{sm_C} - A_{sm_R}) * D_{sm_R}, \\ & A_{sm} = \texttt{yseasmean}(A), \\ & D_{sm} = \texttt{yseasmean}(D), \end{aligned}$$

with

$$\begin{aligned} \Delta &:= \text{weighted difference of albedo in } \frac{W}{m^2} \\ A &:= \text{albedo, quotient of upwelling, downwelling radiation in } \% \\ D &:= \text{downwelling solar radiation in } \frac{W}{m^2} \end{aligned}$$

Fig. 3: **Figure:** part of documentation of a diagnostic method, example

2.2.1 Candidate experiment vga0188 ($\mathbf{C_0}$)

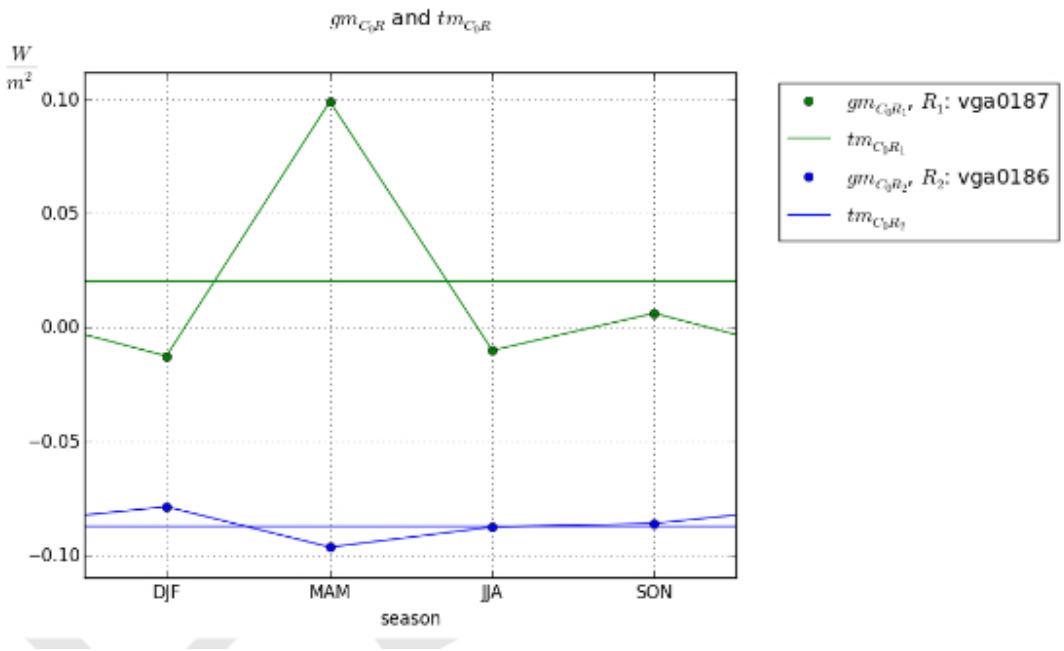
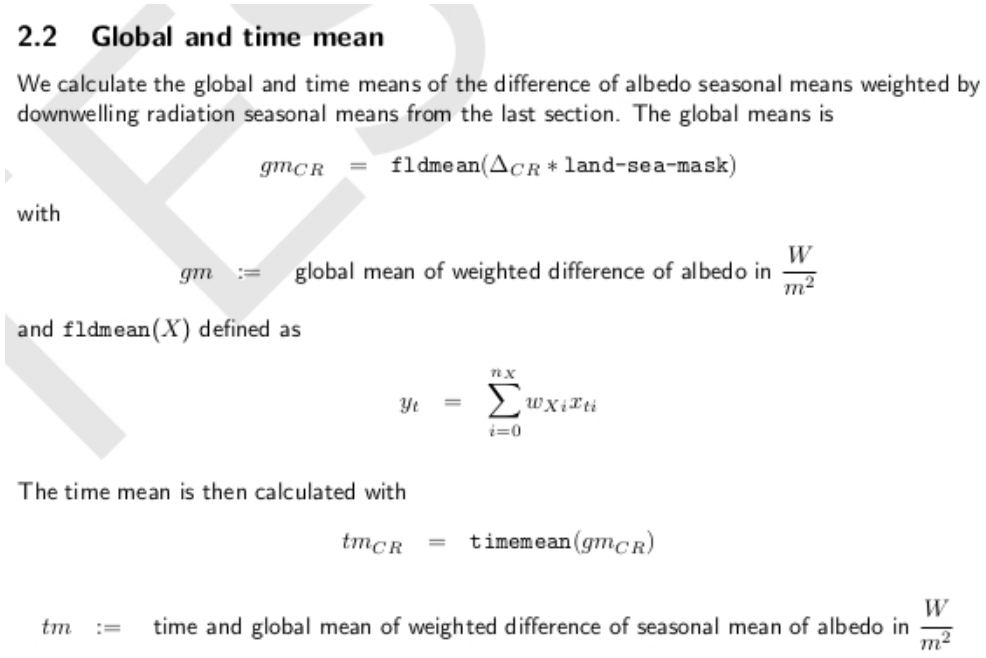


Fig. 4: **Figure:** difference plot of global and time mean for one candidate and two references, example



2.2 Global and time mean

We calculate the global and time means of the difference of albedo seasonal means weighted by downwelling radiation seasonal means from the last section. The global means is

$$gm_{CR} \quad = \quad \texttt{fldmean}(\Delta_{CR} * \texttt{land-sea-mask})$$

with

$$gm \quad := \quad \text{global mean of weighted difference of albedo in } \frac{W}{m^2}$$

and $\texttt{fldmean}(X)$ defined as

$$y_t \quad = \quad \sum_{i=0}^{n_X} w_{Xi} x_{ti}$$

The time mean is then calculated with

$$tm_{CR} \quad = \quad \texttt{timemean}(gm_{CR})$$

$$tm \quad := \quad \text{time and global mean of weighted difference of seasonal mean of albedo in } \frac{W}{m^2}$$

Fig. 5: **Figure:** part of documentation of a diagnostic method, example

Summary

For the evaluation and the closing of the report an overview of all diagnostic is given in a benchmark value table.

field	analyis	unit	reference	candidate		
				vga0188	vga0187	vga0186
albedo	delta_gtd	Wm-2	$\mathbf{R_0}$	-	3.5492e-01	2.4916e-01
			$\mathbf{R_1}$	3.6422e-01	-	2.2999e-01
			$\mathbf{R_2}$	2.5034e-01	2.2253e-01	-
	delta_gtm	Wm-2	$\mathbf{R_0}$	-	-2.3228e-02	8.5496e-02
			$\mathbf{R_1}$	2.0710e-02	-	1.0909e-01
			$\mathbf{R_2}$	-8.7103e-02	-1.0847e-01	-

Fig. 6: **Figure:** summary with three experiments, both as candidates and references, example

Running

Getting Started

Steps

- check out branch from git repository
- install python, pycmbs, ...
- update path variable or change directory

- set up passwordless ssh
- run bench4jsbach.py

directory structure

bin source files

configs specification and templates for configuration files

doc Sphinx documentation source files and Makefile

prod scripts and configurations for diagnostic fields

scripts essential parts of **bench4jsbach**

tests different testing environments

work currently hardcoded working directory

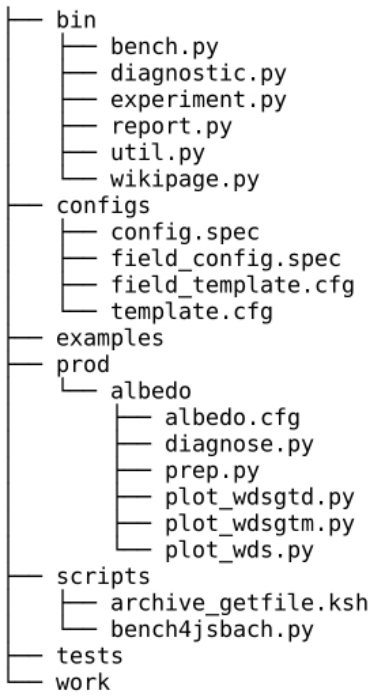


Fig. 7: **Figure:** <mypath>/jsbach/tools/bench4jsbach

Running

Workflow

startup choose a keyname for run and execute **bench4jsbach** with it. This initial run creates

- working environment
- configuration file

adapt generated standard configuration to your requirements

- specify candidate and reference experiments

- use all standart diagnostic included or select some tests
- (developers: supply a new diagnostic field)

run **bench4jsbach** providing the configuration file

- fetches required output files of the experiments
- interpolates the data according to requirements
- runs preprocessing, diagnostics, plotting in loops over experiments
- writes report

study report, if required check other **bench4jsbach** output, rethink, enjoy :)

keep **bench4jsbach** report, delete working directory

Experiments

Candidates and references

- define them by a subsection key
  - you are done for references from wikipage Bench4JSBACH simulations
  - otherwise provide type, path, and host/archive inside the dkrz/zmaw domain
- multiple experiments possible
- experiments could serve both in one run

```
[experiments]
    [[tra0163]]
        @host=mistral@
        @path=/mnt/lustre01/work/mh0469/m212070/Cosmos/mpiesm-1.2.00/experiments@
        type=MPIESM1
        candidate=True
        reference=False
        report=...analyse XXX albedo scheme...
```

Listing: experiment definition in main configuration file `main.cfg`

References

You can find reference experiments on the wikipage Bench4JSBACH simulations.

Reference simulations:

Simulations that have been declared as reference.

simulation name	model type	model version	model mode	atm res	oce res	simulation period	host	data storage	comment
vga0186	MPIESM1			T63	GR15	1950-2099	archive	/hpss/arch/mj0060/m220053/experiments	with spitfire
vga0187	MPIESM1			T63	GR15	1950-1996	archive	/hpss/arch/mj0060/m220053/experiments	with spitfire, permafrost
vga0188	MPIESM1			T63	GR15	1950-2126	archive	/hpss/arch/mj0060/m220053/experiments	control run
slo0010	MPIESM1			T63	GR15	1850-3399	mistral	/mnt/lustre01/work/mh0469/m211032/Cosmos/mpiesm-1.2.00/experiments	

Fig. 8: **Figure:** wikipage Bench4JSBACH simulations, detail

Participate

Yield

- usability for your scientific question
- test your simulation output data
- influence the development of the tool

Introducing Features

Effort

1. For a diagnostic field ...
    - determine the most significant analyses and benchmarks
    - write texts for the report
    - implement the processing chain: preprocessing, analysing, plotting
    - define the configurations files
    - make tests
2. for a simulation/observation type ...
    - locate sources of experiment data
    - index relevant variable output data
    - identify methods to extract it

- characterize temporal and spatial structure
- indentify interpolation methods

Diagnostic Fields

A diagnostic field ...

- clusters scientific processing
- is defined by a specific configuration file
- provides at least one diagnostic method
- should supply a metric/benchmark
- is mapped to one section in the report

```
[fields]
    [[albedo]]
        config_filename=albedo.cfg
```

Listing: main.cfg, example

Its configuration file defines ...

- the field specific part of the interface and the associated scripts
- corresponding workflow
- keys/infixes for I/O interface

Configuration

```
[fetch]
    [[D_select]]
        [[[MPIESM1]]]
            code=21
            submodel=jsbach
            stream=jsbach
    [[U_select]]
        [[[MPIESM1]]]
            code=22
            submodel=jsbach
            stream=jsbach

[interpolation]

[preprocessing]
    exec=prep.ex
    input=D_select,U_select,
    [[UD_div]]
    [[A_seasmean]]
    [[D_seasmean]]
                                ...
```

```
...
[diagnostic]
    exec=diagnostic.ex
    input=A_seasmean,D_seasmean,
    [[alb_wds]]
    [[alb_wdsgm]]
    [[alb_wdsgtm]]
        metric=True

[plotting]
    suffix=png
    [[wds_plot]]
       exec=plot_wds.ex
       input=alb_wds,
    [[wdsgtm_plot]]
       exec=plot_wdsgtm.ex
       input=alb_wdsgm,alb_wdsgtm,

[report]
```

Listing: albedo.cfg, example

I/O Dependencies

Following the link from the listing shown above leads to an illustration of the I/O deplendencies.

Processing

Scripts

- each either for preprocessing, diagnostic or plotting
- work on local data, interpolated as indicated by user and modeltype
- layed out for one candidate and one reference experiment
- run by **bench4jsbach** in loops over multiple experiments
- interface has to be provided by the respective and the main configuration file concerning
  - input and output filenames and format
  - grid, spatial resolution, tiling and global fragmentation
  - time resolution and period
  - other
- called in order of I/O dependencies
- written in any programming language

Runtime

Fetching Data

Specialized experiment type handles ...

- provide output streams from
  - remote server via passwordless ssh
  - archive by means of pftp, storing it to `downloads`-directory
- select variables and periods needed for diagnostic
  - using e.g. **CDO**
  - write it to `processed`-directory using key/infix from configuration

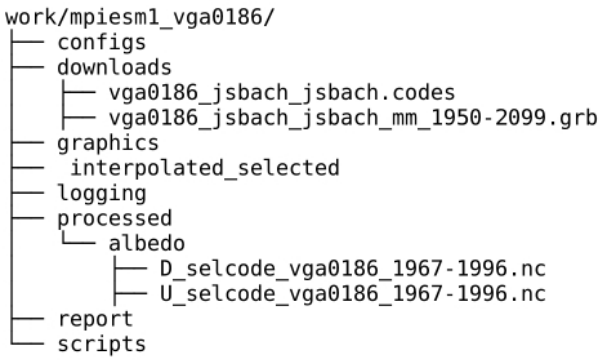


Fig. 9: **Figure:** working directory expanded to focus attention on fetched and selected data, example

Configuration

Configuration API Reference

The configuration files for a **bench4jsbach** run are validated with corresponding specification files to check integrity and types of items and values.

Specification for the main configuration

```
################################################################################
# generated automatically
################################################################################

bench_name=string()                             # AUTO: name of b4j run
author=string()                                 # AUTO: username
git_repository=string_list(default=None)        # AUTO: b4j repository
git_branch=string(default=None)                 # AUTO:
git_sha=string(default=None)                    # AUTO:
git_status=option('clean','modified',None)      # AUTO:
redmine_url=string()                            # AUTO: b4j redmine site
redmine_project=string()                        # AUTO:
redmine_wikipage=string()                       # AUTO:
host_name=string(default=None)                  # AUTO: host of b4j run
host_ipaddress=string(default=None)             # AUTO:
b4j_base=string()                               # AUTO:
timestamp=string()                              # AUTO: b4j run config file creation
                                                # AUTO:   time

################################################################################
# edited by user
################################################################################

redmine_user=string()                           # USER: user redmine login
redmine_key=string()                            # USER: to find in redmine,
                                                # USER:    "My account", right panel,
                                                # USER:    "Api access key"
institute=string()                              # USER: user institute
department=string()                             # USER: user department
reporting=boolean()                             # USER: write report for b4j run,
                                                # USER:     default "True"
```

```
[experiments]                                   # USER: section experiments for b4j run
[[__many__]]                                    # USER: subsection <A>, <A>:= id for one
                                                # USER:     experiment
name_orig=string()                              # USER: <A>: original infix/id
exp_type=option('CMIP5','MPIESM1')              # USER: <A>: experiment type
path=string(default=None)                       # USER: <A>: location
host=string(default=None)                       # USER: <A>:
iscan=boolean()                                 # USER: <A>: candidate role
isref=boolean()                                 # USER: <A>: reference role
startyear=string()                              # USER: <A>: startyear of <A> in b4j run
endyear=string()                                # USER: <A>: endyear of <A> in b4j run
initialyear=string(default=None)                # USER: <A>: initialyear of experiment
finalyear=string(default=None)                  # USER: <A>: finalyear of experiment

[fields]                                        # USER: section diagnostic fields for
                                                # USER:     b4j run
[[__many__]]                                    # USER: subsection <F>, <F>:= id for
                                                # USER:     one diagnostic field
config_filename=string()                        # USER: <F>: configuration file's
                                                # USER:     relative path from b4j_base
                                                # USER:     or filename in
                                                # USER:     <b4j_base>/prod/scripts
```

Listing: Specification file `b4j_config.spec`

Specification for the diagnostic field configurations

```
field_name=string()                             # USER: field id conform to id/name
                                                # USER:     of diagnostic field <F>
                                                # USER:     in main configuration
suffix=string()                                 # USER: output format suffix for
                                                # USER:     processed data of b4j run
frequency=option('yearly','monthly',
                 '6-hourly',
                 default='monthly')             # USER: output frequence to use in
                                                # USER:     b4j run
description=string()                            # USER: description

[fetch]                                         # USER: section fetch for diagnostic
                                                # USER:     field
[[__many__]]                                    # USER: subsection <V>,
                                                # USER:     <V>:= id/infix for one
                                                # USER:     variable file to fetch
longname=string(default=None)                   # USER: <V>: descriptive name
unit=string()                                   # USER: <V>: physical unit
[[[__many__]]]                                  # USER: <V>: subsubsection <VM>:
                                                # USER:     experiment type conform
                                                # USER:     to type of experiment in
                                                # USER:     main configuration
name=string()                                   # USER: <VM>: name
code=string()                                   # USER: <VM>: code
submodel=string(default=None)                   # USER: <VM>: submodel
stream=string(default=None)                     # USER: <VM>: stream

[preprocessing]                                 # USER: section preprocessing for
                                                # USER:     diagnostic field
exe=string()                                    # USER: executable script to call
                                                # USER:     in b4j run
```

```
args=string_list                                # USER: input files needed by the
                                                # USER:     script
[[__many__]]                                    # USER: id/infix for output file
                                                # USER:     written by the script

[diagnose]                                      # USER: section diagnose for
                                                # USER:     diagnostic field
exe=string()                                    # USER: executable script to call in
                                                # USER:     b4j run
args=string_list                                # USER: input files needed by the
                                                # USER:     script
[[__many__]]                                    # USER: id/infix <D> for output file
                                                # USER:     written by the script

[plotting]                                      # USER: section plotting for
                                                # USER:     diagnostic field
suffix=string()                                 # USER: output format's suffix
[[__many__]]                                    # USER: subsection <O>,
                                                # USER:     <O>:= id of plot script
exe=string()                                    # USER: <O>: executable script
                                                # USER:     to call in b4j run
args=string_list                                # USER: <O>: input files needed by
                                                # USER:     the script
nref=option('single','multiple')                # USER: <O>: as indicated by plot:
                                                # USER:     comparison with single
                                                # USER:     or all reference experiments
plot_title=string_list(default=None)            # USER: <O>: title string for plot
[[[__many__]]]                                  # USER: <O>: id/infix for output
                                                # USER:     file written by the script

[report]                                        # USER: section report for diagnostic
                                                # USER:     field
title=string(default=None)                      # USER: title for the diagnostic
                                                # USER:     field's section in the report
abstract=string(default=None)                   # USER: scientific description of the
                                                # USER:     diagnostic field
technical_TeXed=string(default=None)            # USER: technical description of the
                                                # USER:     diagnostic field
[[__many__]]                                    # USER: subsection <SD> for
                                                # USER:     subsection for a
                                                # USER:     diagnostic method,
                                                # USER:     key <SD> conform to key <D>
title=string(default=None)                      # USER: <SD>: title of report subsection
name=string()                                   # USER: <SD>: name of diagnostic
                                                # USER:     method
plottype=option('map','metric','barchart')      # USER: <SD>: type of plot
isbench=boolean()                               # USER: <SD>: diagnostic method is
                                                # USER:     used as benchmark
benchvar=string(default=None)                   # USER: <SD>: name of displayed
                                                # USER:     variable in plotfile
benchunit=string(default=None)                  # USER: <SD>: physical unit
ismatrix=boolean()                              # USER: <SD>: comparison with all
                                                # USER:     references
isvector=boolean()                              # USER: <SD>: comparison pairwise
abstract=string(default=None)                   # USER: <SD>: scientific description
                                                # USER:     of diagnostic method
technical_TeXed=string(default=None)            # USER: <SD>: technical description
                                                # USER:     of diagnostic method
```

Listing: Specification file `b4j_field_config.spec`

Example configuration for the report section of the diagnostic field ‘albedo´

```
[report]
    title="diagnostic and benchmarks"
    abstract="""The analysis and the benchmarking ..."""
    [[albedo_wds]]
        title="Weighted difference of seasonal means"
        name=delta
        type=matrix
        technical_teXed="""The difference between candid ..."""
        abstract="""The four figures provide an overview ..."""
```

Listing: Example configuration file math:*mathtt{albedo.cfg}*, abridged

2 Albedo: diagnostic and benchmarks

The analysis and the benchmarking of the land surface albedo aims at elaborating the climate relevance of the albedo differences between the candidate experiment and the reference dataset. Therefore, signals in net surface solar radiation that result from differences in land surface albedo (instead of signals in surface albedo itself) are shown.

2.1 Seasonal mean difference

The difference between candidate and reference experiments albedo seasonal means is weighted with the reference experiments downwelling radiation seasonal means:

$$\begin{aligned} \Delta_{CR} &= (A_{sm_C} - A_{sm_R}) * D_{sm_R}, \\ & A_{sm} = \texttt{yseasmean}(A), \\ & D_{sm} = \texttt{yseasmean}(D), \end{aligned}$$

with

$$\begin{aligned} \Delta &:= \text{ weighted difference of albedo in } \frac{W}{m^2} \\ A &:= \text{ albedo, quotient of upwelling, downwelling radiation in } \% \\ D &:= \text{ downwelling solar radiation in } \frac{W}{m^2} \end{aligned}$$

and $\texttt{yseasmean}(X)$ defined as

$$\begin{aligned} y_{SX} &= n_S^{-1} \sum_{i=0}^{n_S} x_{iX}, \\ & t \mapsto i, t \in S, \\ & S \in \{\text{DJF, MAM, JJA, SON}\}. \end{aligned}$$

The four figures provide an overview of seasonal changes as well as regional changes. The displayed signals in time averaged net surface solar radiation [Wm-2] can be compared with other forcings that impact the local climate via the surface energy balance (e.g. changes in surface heat fluxes or changes in surface thermal radiation). A local absolute signal of $y <$ 3Wm-2 is small and an absolute signal of $>$ 10Wm-2 is probably changing the local climate significantly.

Fig. 10: **Figure:** Resulting albedo section in the report

Remarks on report configuration

- report related items are gathered at the end of configuration files to improve readability
- mapping via key correspondence with diagnostic sections

Advantage of static report texts

- documentation on board
- evaluation criteria for benchmarks supplied directly

Indices and tables

- genindex
- modindex
- search